Лабораторная работа №1
Модульное тестирование

Цель работы

Освоить базовые навыки применения модульного тестирования с акцентом на выявление дефектов и оценку качества реализации программных компонентов.

Теоретическая часть

Модульное тестирование – это процесс верификации отдельных компонентов программной системы (классов, методов, функций) на корректность их поведения в изоляции от остальных частей приложения. Основная цель – подтверждение того, что каждый модуль удовлетворяет своему функциональному назначению, включая обработку как штатных, так и нештатных ситуаций. Тесты должны быть независимыми: результат выполнения одного теста не должен зависеть от других, а время выполнения каждого теста должно быть минимальным. Для обеспечения изоляции тестируемого компонента от внешних зависимостей (например, баз данных, сетевых сервисов, системного времени) применяются заглушки (*stubs*), мок-объекты (*mocks*) и фейковые реализации (*fakes*).

Ключевые принципы построения модульных тестов

1. Типы тестовых сценариев

- **Позитивные тесты** проверяют корректную работу модуля при валидных входных данных. Например, проверка метода валидации email-адреса на строке `"user@example.com"`.
- **Негативные тесты** проверяют реакцию модуля на некорректные или недопустимые входные данные. Например, передача `null` или пустой строки в метод, требующий непустой параметр.
- **Граничные случаи** исследуют поведение модуля на предельных значениях входных параметров. Например, для метода, принимающего целочисленные значения в диапазоне от 1 до 100, следует проверить значения 0, 1, 99, 100 и 101.
- **Тесты на исключения** явно проверяют выброс ожидаемых исключений в определённых условиях.

2. Подход «чёрного ящика» при разработке тестов

При тестировании «чёрного ящика» тестирующий рассматривает модуль как неделимую систему, не имея доступа к его внутренней структуре. Тесты разрабатываются исключительно на основе спецификации интерфейса: входных данных, ожидаемых выходных данных и описанных исключений. Такой подход позволяет проверить соответствие реализации требованиям и является основой для большинства модульных тестов.

Важно: тесты «чёрного ящика» разрабатываются **до** или **параллельно** с реализацией, поскольку они основаны на спецификации, а не на коде.

3. Подход «белого ящика» при разработке тестов

В отличие от «чёрного ящика», тестирование «белого ящика» предполагает знание внутренней структуры тестируемого модуля. Цель таких тестов – обеспечить покрытие всех логических путей, ветвлений, циклов и состояний, которые могут быть не охвачены

тестами «чёрного ящика». Тесты «белого ящика» дополняют набор тестов «чёрного ящика», выявляя скрытые ошибки, такие как:

- Некорректная обработка внутренних состояний;
- Пропущенные ветви условий (`if/else`);
- Неправильная инициализация полей;
- Нарушение инвариантов класса.

Важно: тесты «белого ящика» разрабатываются **после** анализа реализации и служат для расширения покрытия, а не для замены тестов «чёрного ящика».

4. Принцип AAA (Arrange–Act–Assert)

- **Arrange** – подготовка окружения: создание объектов, инициализация данных, настройка заглушек.
- **Act** – вызов тестируемого метода.
- **Assert** – проверка результата с помощью утверждений (`assertions`).

Пример: для метода сложения чисел создаётся экземпляр калькулятора (Arrange), вызывается `Add(2, 3)` (Act), проверяется, что результат равен 5 (Assert).

5. Принцип FIRST

Хорошие модульные тесты обладают следующими качествами:

- **Fast** – быстрые (выполняются за миллисекунды);
- **Independent** – независимые (не зависят от порядка запуска);
- **Repeatable** – повторяемые (дают одинаковый результат в любом окружении);
- **Self-validating** – самопроверяемые (результат определяется автоматически);
- **Timely** – своевременные (пишутся до или сразу после реализации).

6. Пирамида тестирования

Модульные тесты составляют основу пирамиды (~70% всех тестов). Интеграционные тесты (~20%) проверяют взаимодействие компонентов, а end-to-end тесты (~10%) – сквозные пользовательские сценарии. Такое распределение обеспечивает оптимальный баланс между надёжностью и затратами на поддержку.

7. Изоляция зависимостей

Для чистого модульного тестирования необходимо устранить влияние внешних зависимостей. Рекомендуемые практики:

- Внедрение зависимостей через интерфейсы (Dependency Injection);
- Использование `IDateTimeProvider` вместо прямого обращения к `DateTime.Now`;
- Применение моков и заглушек через фреймворки (Moq, NSubstitute и др.).

Практическая часть

В данной лабораторной работе используется язык C# и фреймворк NUnit. Для проверок рекомендуется использовать синтаксис на основе ограничений (`Assert.That(..., Is.EqualTo(...))`), который обеспечивает гибкость и читаемость. Однако следует отметить, что конструкции вроде `Assert.AreEqual` **не являются устаревшими** и допустимы в NUnit; выбор стиля зависит от принятых соглашений проекта.

Для получения тестируемого кода могут быть использованы **генеративные ИИ-системы**, доступные в Российской Федерации без необходимости использования VPN или специальных технических средств.

Доступные в России генеративные ИИ (без дополнительных манипуляций):

- **Qwen** – https://qwen.ai
- **GigaChat** – https://gigachat.ru
- **DeepSeek** – https://deepseek.com (часто доступен через зеркала)
- **YandexGPT** – https://yandex.ru/yandexgpt
- **Sber AI / Salute** – https://salute.technology
- **Llama.cpp (локальный запуск)** – https://github.com/ggerganov/llama.cpp
- **LM Studio (локальный запуск)** – https://lmstudio.ai

Рекомендация: при выполнении лабораторной работы используйте как минимум три различные модели из списка выше, чтобы сравнить качество генерации.

Для получения корректной реализации необходимо сформулировать точный промпт, например:

Сгенерируй класс [Название] на C# с методами:
[Метод1]: [описание], входные параметры: [типы], возвращает: [тип].
Особые условия: [валидация, исключения].
Класс должен реализовывать интерфейс I[Название], находящийся в пространстве имён Lab.Interfaces.

Создание структуры проекта

Выполните в терминале:

```
dotnet new nunit -n Lab
cd Lab
mkdir Interfaces Implementations/GenCode1 Implementations/GenCode2
Implementations/GenCode3
```

Создайте интерфейс IToDoListManager.cs в папке Interfaces:

```
namespace Lab.Interfaces;
public interface IToDoListManager
{
    void AddTask(string description);
    void RemoveTask(int id);
    void MarkTaskAsCompleted(int id);
    string[] GetTasks();
}
```

Генерация реализации

Промпт для нейросети:

Создай класс ToDoListManager, реализующий интерфейс IToDoListManager.
Интерфейс объявлен в пространстве имён Lab.Interfaces.
Класс должен находиться в пространстве имён Lab.Implementations.GenCode1.

Сгенерированный код сохраняется в файл Implementations/GenCode1/ToDoListManager.cs:

```
using Lab.Interfaces;
namespace Lab.Implementations.GenCode1;
public class ToDoListManager : IToDoListManager
{
    private class TaskItem
    {
        public int Id { get; set; }
        public string Description { get; set; } = string.Empty;
        public bool IsCompleted { get; set; }
        public override string ToString() =>
```

```
            $"[{(IsCompleted ? "X" : " ")}] {Id}: {Description}";
    }

    private readonly List<TaskItem> _tasks = new();
    private int _nextId = 1;

    public void AddTask(string description)
    {
        if (string.IsNullOrWhiteSpace(description))
            throw new ArgumentException("Описание задачи не может быть
пустым.", nameof(description));
        _tasks.Add(new TaskItem
        {
            Id = _nextId++,
            Description = description.Trim(),
            IsCompleted = false
        });
    }

    public void RemoveTask(int id)
    {
        var task = _tasks.FirstOrDefault(t => t.Id == id);
        if (task == null)
            throw new ArgumentException($"Задача с ID {id} не найдена.",
nameof(id));
        _tasks.Remove(task);
    }

    public void MarkTaskAsCompleted(int id)
    {
        var task = _tasks.FirstOrDefault(t => t.Id == id);
        if (task == null)
            throw new ArgumentException($"Задача с ID {id} не найдена.",
nameof(id));
        task.IsCompleted = true;
    }

    public string[] GetTasks()
    {
        return _tasks
            .OrderBy(t => t.Id)
            .Select(t => t.ToString())
            .ToArray();
    }
}
```

Написание тестов

Файл Tests/UnitTest1.cs:

```
using NUnit.Framework;
using Lab.Interfaces;
using Lab.Implementations.GenCode1;
using System.Linq;
namespace Tests;
[TestFixture(Category = "GenCode1")]
public class GenCode1_BlackBoxAndWhiteBoxTests
{
    private IToDoListManager _manager = new ToDoListManager();
```

```
    // ---------- Тесты "чёрного ящика" (на основе спецификации) ---------
-

    [Test]
    public void AddTask_ValidInput_AddsTaskCorrectly()
    {
        _manager.AddTask("Купить хлеб");
        var tasks = _manager.GetTasks();
        Assert.That(tasks, Has.Length.EqualTo(1));
        Assert.That(tasks[0], Does.Contain("Купить хлеб"));
    }

    [Test]
    public void AddTask_EmptyString_ThrowsArgumentException()
    {
        var ex = Assert.Throws<ArgumentException>(() =>
_manager.AddTask(""));
        Assert.That(ex!.Message, Does.StartWith("Описание задачи не может
быть пустым."));
    }

    [Test]
    public void RemoveTask_NonExistentId_ThrowsArgumentException()
    {
        var ex = Assert.Throws<ArgumentException>(() =>
_manager.RemoveTask(999));
        Assert.That(ex!.Message, Does.Contain("не найдена"));
    }

    [Test]
    public void MarkTaskAsCompleted_ValidId_MarksTask()
    {
        _manager.AddTask("Протестировать код");
        _manager.MarkTaskAsCompleted(1);
        var tasks = _manager.GetTasks();
        Assert.That(tasks[0], Does.StartWith("[X] 1:"));
    }

    // ---------- Тесты "белого ящика" (на основе анализа реализации) ----
------

    [Test]
    public void AddTask_WhitespaceOnly_ThrowsArgumentException()
    {
        // Этот сценарий может быть упущен в спецификации, но реализация
использует IsNullOrWhiteSpace
        var ex = Assert.Throws<ArgumentException>(() =>
_manager.AddTask("   \t\n  "));
        Assert.That(ex!.Message, Does.StartWith("Описание задачи не может
быть пустым."));
    }

    [Test]
    public void IdIncrement_PersistsAfterRemoval()
    {
        // Проверка внутреннего состояния: ID не должен переиспользоваться
        _manager.AddTask("A");
        _manager.AddTask("B");
        _manager.RemoveTask(1);
        _manager.AddTask("C");
        var tasks = _manager.GetTasks();
```

```
        // ID: 2 (B), 3 (C)
        Assert.That(tasks.Select(t => t.Split(':')[0]),
                    Is.EquivalentTo(new[] { "[ ] 2", "[ ] 3" }));
    }

    [Test]
    public void GetTasks_ReturnsOrderedById()
    {
        // Проверка внутренней логики сортировки
        _manager.AddTask("Z");
        _manager.AddTask("A");
        _manager.AddTask("M");
        var tasks = _manager.GetTasks();
        Assert.That(tasks[0], Does.StartWith("[ ] 1:"));
        Assert.That(tasks[1], Does.StartWith("[ ] 2:"));
        Assert.That(tasks[2], Does.StartWith("[ ] 3:"));
    }
}
```

Примечание: тесты «чёрного ящика» разрабатываются **до** анализа кода и проверяют соответствие спецификации. Тесты «белого ящика» добавляются **после** изучения реализации и направлены на выявление недостатков, не охваченных спецификацией (например, обработка пробельных строк, поведение счётчика ID, порядок вывода).

Запуск тестов

```
dotnet test --logger "console;verbosity=normal"
```

Если некоторые тесты не проходят, необходимо:

1. Убедиться, что тест корректен (особенно тесты «белого ящика»);
2. Проанализировать, является ли ошибка следствием некорректной реализации;
3. В отчёте описать выявленные дефекты, **не исправляя** сгенерированный код.

Индивидуальные задания (расширенный список до 25 тем)

Каждое задание предполагает создание **одного класса с 2–4 методами**, не требующего внешних зависимостей (баз данных, сети и т.п.), но содержащего логику, подлежащую тестированию.

1. **EmailValidator** – проверка корректности email и извлечение домена
2. **TemperatureConverter** – преобразование между °C, °F, K
3. **RomanNumeralConverter** – арабские ↔ римские числа
4. **UrlSanitizer** – очистка и нормализация URL
5. **PasswordStrengthChecker** – оценка надёжности пароля
6. **FileNamer** – генерация безопасных имён файлов
7. **CsvRowParser** – разбор строки CSV с кавычками
8. **ColorParser** – HEX ↔ RGB ↔ название цвета
9. **PhoneNumberFormatter** – нормализация телефонных номеров к формату +7 (XXX) XXX-XX-XX
10. **JsonPathExtractor** – извлечение значения по простому JSON-пути (без библиотек)
11. **BinaryToDecimalConverter** – преобразование двоичной строки в число и обратно
12. **PalindromeChecker** – проверка слова/фразы на палиндром с учётом регистра и пробелов
13. **IpAddressValidator** – валидация IPv4-адреса
14. **CreditCardValidator** – проверка номера карты по алгоритму Луна

15. **Base64Encoder** – кодирование/декодирование строки в Base64 без использования стандартных библиотек
16. **StringCompressor** – простое RLE-сжатие строк (например, "aaabbc" → "a3b2c1")
17. **DateParser** – парсинг даты из строки в форматах DD.MM.YYYY, YYYY-MM-DD
18. **MathExpressionEvaluator** – вычисление простого арифметического выражения (без скобок)
19. **UniqueIdentifierGenerator** – генерация уникального ID на основе времени и счётчика
20. **TextTruncator** – обрезка текста до N символов с добавлением "..."
21. **CaseConverter** – преобразование между camelCase, PascalCase, snake_case
22. **FractionReducer** – сокращение дроби до несократимого вида
23. **MorseCodeTranslator** – текст ↔ азбука Морзе
24. **VowelCounter** – подсчёт гласных в строке с учётом русского и английского алфавитов
25. **BracketsBalancer** – проверка сбалансированности скобок (), [], {}

Все модули должны быть **чистыми функциями или stateless-классами**, не зависящими от внешнего окружения.

Требования к отчёту

1. **Техническое задание и промпт**
   – Исходное описание задачи.
   – Точный текст промпта.
2. **Исходный код**
   – Полные реализации от трёх нейросетей.
   – Краткий анализ различий.
3. **Тесты**
   – Сценарии тестов, в которых для каждого теста должно быть указано, что проверяется, какой используется тестовый набор данных и какой ожидается результат.
   – Код тестов с комментариями, разделёнными на «чёрный» и «белый» ящик.
   – Таблица результатов тестирования всех реализаций кода.
   – Анализ причин падения тестов.
4. **Выводы**
   – Оценка качества (1–5) для каждой реализации.
   – Как тестирование помогло найти ошибки?
   – Как улучшить промпт?

Контрольные вопросы

1. Почему модульные тесты должны быть изолированными? Как добиться изоляции при тестировании метода, зависящего от системного времени?
2. Какую стратегию вы выберете для тестирования кода, сгенерированного нейросетью: сначала писать тесты «чёрного ящика», а потом запрашивать реализацию, или наоборот? Обоснуйте.
3. В каких случаях негативные тесты (проверка некорректных входных данных) важнее позитивных? Приведите пример из вашей лабораторной работы.

4. Как вы обнаружите «ложноположительный» тест (тест проходит, но код работает некорректно)? Опишите технику проверки.
5. Этический аспект: какие риски возникают, если использовать нейросетевую генерацию кода в критически важных системах (медицина, финансы)? Как тестирование снижает эти риски?
6. Помимо покрытия кода (code coverage), какие метрики качества тестов вы бы предложили для оценки надежности сгенерированного нейросетью решения?
7. После запуска тестов вы обнаружили, что 30% сценариев падают. Какие шаги вы предпримете: исправите тесты, перепишете промпт для нейросети или измените архитектуру кода?